

RELATÓRIO TÉCNICO: CONEXÕES SEGURAS

1. INTRODUÇÃO

Este relatório apresenta os resultados da Parte 1 do Projeto "Conexões Seguras", analisando as vulnerabilidades de infraestrutura em Camadas 2 e 3 (Sniffing), defesas na Camada de Transporte (TLS/HTTPS), segurança de sessão na Camada de Aplicação e o isolamento de tráfego por Redes Privadas Virtuais (VPNs). O objetivo é demonstrar a interdependência dos mecanismos de segurança na proteção de dados corporativos através de fundamentos teóricos e Provas de Conceito (PoCs).

2. MÓDULO 1: SNIFFING E INTERCEPTAÇÃO (CAMADA 2 E 3)

2.1 Fundamentação Teórica: Modo Promíscuo e a NIC

No modo padrão (*Não-Promíscuo*), a Placa de Interface de Rede (NIC) atua como um filtro de hardware na Camada 2 (Enlace) do modelo OSI. Ela examina o Endereço MAC de destino de cada quadro e descarta imediatamente os que não são destinados ao próprio host (exceto tráfego legítimo de broadcast ou multicast), poupando processamento da CPU.

Quando configurada em **Modo Promíscuo**, esse filtro por hardware é desativado. A NIC aceita e encaminha todo o tráfego do meio físico para o sistema operacional. Num cenário de ameaça interna, um atacante pode usar este modo para realizar *Sniffing*, capturando dados confidenciais que trafeguem pela rede local (LAN) sem criptografia.

2.2 Demonstração Prática (PoC)

- **Ferramenta:** Wireshark v4.x.
- **Execução:** 1. Inicializou-se o Wireshark com privilégios de administrador (sudo).

2. Ativou-se a opção "*Enable promiscuous mode on all interfaces*" nas configurações de captura.

3. Selecionou-se a interface ativa e iniciou-se a captura.

4. Num terminal isolado, gerou-se tráfego com o comando ping para o gateway da rede.

5. Interrompeu-se a captura e aplicou-se o filtro icmp.

- **Resultado:** O software capturou com sucesso os pacotes ICMP (*Echo Request* e *Echo Reply*) entre os nós da rede, demonstrando que o tráfego em texto limpo no meio físico fica vulnerável à interceptação quando o modo promíscuo está ativo.

3. MÓDULO 2: CRIPTOGRAFIA E HANDSHAKE TLS

3.1 Fundamentação Teórica: Criptografia Híbrida

O HTTPS utiliza **Criptografia Híbrida** para equilibrar segurança robusta e desempenho. Usar apenas criptografia assimétrica (chaves pública/privada como RSA ou ECC) para todo o volume de dados tornaria a web inviável: os cálculos matemáticos complexos exigiriam processamento massivo da CPU, gerando alta latência e exaustão dos servidores.

A abordagem híbrida divide o processo em duas fases:

1. **Fase Assimétrica (Handshake):** O cliente solicita a conexão; o servidor envia o seu Certificado Digital (X.509) com a sua chave pública. O cliente gera uma **Chave de Sessão** efêmera (aleatória), cifra-a com a chave pública do servidor e envia-lha. O servidor decifra-a com a sua chave privada. Ambos possuem agora a mesma chave de sessão de forma segura.
2. **Fase Simétrica (Transferência de Dados):** Com a chave de sessão compartilhada, a fase assimétrica fecha. O tráfego subsequente é encriptado por algoritmos simétricos rápidos (como o **AES**), que operam por transposição e substituição de bits em blocos, exigindo baixo processamento e garantindo alta velocidade.

3.2 Demonstração Prática (PoC)

- **Ferramenta:** DevTools (Google Chrome).
- **Execução:** Acedeu-se ao site <https://github.com/>, abriu o console (F12) e inspecionou o painel *Security* -> *Connection*.
- **Resultado:** O navegador indicou o uso do protocolo TLS 1.3 com a suíte **ECDHE_RSA** para a troca assimétrica de chaves e **AES_128_GCM** para a encriptação simétrica, provando a eficiência da arquitetura híbrida em ambiente real.

4. MÓDULO 3: SEGURANÇA NA CAMADA DE APLICAÇÃO

4.1 Gestão de Sessão e Cookies

O protocolo HTTP é *stateless* (não guarda estado). Para manter o utilizador autenticado, as aplicações utilizam o **Cookie SessionID**. Após o login, o servidor gera um identificador único de alta entropia (SessionID), enviado ao navegador pelo cabeçalho Set-Cookie. O navegador anexa este cookie automaticamente em todas as requisições seguintes.

Como o SessionID equivale à identidade do utilizador, a sua interceptação permite o ataque de **Session Hijacking (Sequestro de Sessão)**, onde o invasor injeta o cookie no seu próprio navegador e assume a conta da vítima, contornando senhas e múltiplos fatores de autenticação (2FA).

Para mitigar o risco, os desenvolvedores devem aplicar as seguintes diretivas:

- **Entropia Criptográfica:** Geração de tokens imprevisíveis por geradores CSPRNG.
- **Flag HttpOnly:** Bloqueia o acesso ao cookie via scripts no navegador (como `document.cookie`), impedindo roubos por falhas de Cross-Site Scripting (XSS).
- **Flag SameSite:** Restringe o envio do cookie em requisições de origens cruzadas, mitigando ataques de Cross-Site Request Forgery (CSRF).

4.2 O Papel do HTTPS e o Risco do HTTP

No HTTPS, todo o cabeçalho HTTP (incluindo os cookies) viaja encriptado pelo túnel TLS. No HTTP convencional, os dados viajam em texto limpo e podem ser capturados diretamente por um sniffer em modo promíscuo (como visto no Módulo 1). Os programadores usam a flag **Secure** para obrigar o navegador a transmitir o cookie apenas em ligações HTTPS protegidas.

4.3 Demonstração Prática (PoC)

- **Ferramenta:** Painel de Armazenamento do DevTools.
- **Execução:** Portal ead do senai, abriu-se o DevTools (F12) e examinou-se a aba *Application* -> *Cookies*.
- **Resultado:** Os cookies críticos de sessão exibiram as flags **HttpOnly** e **Secure** ativas, confirmando que a plataforma impede a leitura do token por JavaScript e proíbe a sua transmissão por canais inseguros.

5. MÓDULO 4: VPN (VIRTUAL PRIVATE NETWORK)

5.1 Mecanismos de Tunelamento e Proteção

Uma VPN estende o perímetro de segurança corporativo através da internet pública. O seu funcionamento baseia-se no **Tunelamento**: o pacote de dados original da rede privada (com IPs internos e payload) é encapsulado como dado puro dentro de um novo pacote de rede externo com IPs públicos e roteáveis para trafegar na Internet.

A segurança do túnel baseia-se em dois pilares:

- **Confidencialidade:** O pacote interno é encriptado com algoritmos simétricos robustos (AES-256 ou ChaCha20) antes do encapsulamento, tornando-o ilegível para terceiros.
- **Integridade:** Aplicam-se códigos baseados em hash (HMAC) ou assinaturas criptográficas. O destino recalcula o hash; se houver divergência de bits por falha ou manipulação, o pacote é descartado.

5.2 Comparativo: OpenVPN vs. IPsec

Critério	Protocolo OpenVPN	Protocolo IPsec
Camada OSI	Opera na Camada 4 (Transporte), sobre UDP ou TCP.	Opera de forma nativa na Camada 3 (Rede).
Segurança	Baseia-se na biblioteca OpenSSL e herda os recursos do TLS (ex: AES-GCM).	Utiliza os cabeçalhos AH (Autenticação/Integridade) e ESP (Confidencialidade).
Flexibilidade	Alta. Usa portas customizáveis (padrão UDP 1194), contornando facilmente Firewalls e NAT.	Menos flexível em NATs restritivos, mas possui alto desempenho processado por hardware (ASICs).
Uso Ideal	Acesso remoto de utilizadores finais (<i>Client-to-Site</i>).	Interconexão permanente de infraestruturas (<i>Site-to-Site</i> / Filiais).

6. CONCLUSÃO

Os testes demonstraram a complementaridade das defesas em rede. A vulnerabilidade de interceptação passiva por Modo Promíscuo (Camada 2/3) é mitigada de forma robusta pelas camadas superiores: a criptografia híbrida do TLS (Camada de Transporte) encripta os dados da rede, as flags nos cookies (Camada de Aplicação) protegem a integridade das sessões dos utilizadores, e o tunelamento por VPN garante o tráfego isolado de dados corporativos através de infraestruturas públicas.